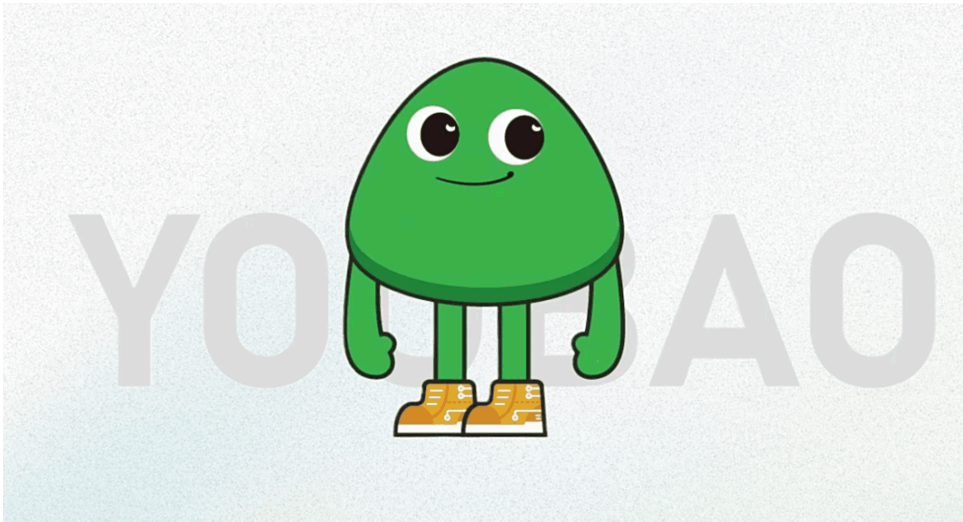


邮宝 YouBao —— 自主渗透 Agent 技术方案

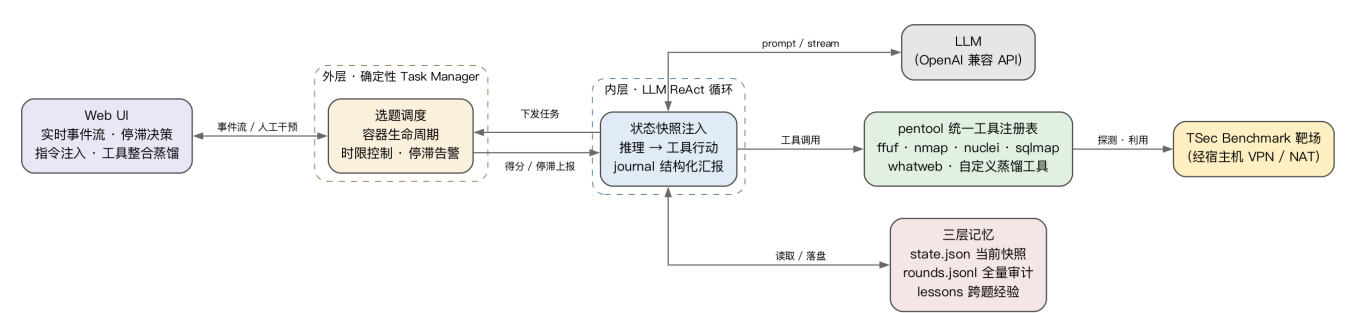


1. 项目概述

邮宝 YouBao 是一个面向 TSec Benchmark 安全靶场的自主渗透测试 Agent。给定一台靶机（或一组靶题），Agent 能够无人值守地完成「侦察 → 工具选择 → 脚本编写 → 漏洞利用 → 提交 flag」的全流程；同时提供 Web UI 支持实时观测与人工接管。系统内核基于 MIT 许可的 nano-pi 极简 coding agent（5 个文件、600+ 行 TypeScript）二次开发，技术栈为 TypeScript 5 / Node.js ≥22，零运行时依赖，通过 OpenAI 兼容 API 接入大模型（默认 deepseek），并以一体化 Dockerfile（内置 ffuf、nuclei 等渗透工具）交付。

2. 技术架构

系统采用「确定性外层 + 概率性内层」的双层 ReAct 架构，核心思想是：把可靠的事交给代码做，把不确定的事交给 LLM 做。



外层: Task Manager (runner.ts, 确定性调度)
VPN 预检 / 选题 / worker 池 / 时限 / 停滞告警

内层: LLM ReAct 循环 (agent.ts)

流式回复 → tool_call 执行 → 结果回灌 context
工具层: run_bash / write_file / edit pentool 注册表 / benchmark API / 蒸馏工具
记忆层: state.json / rounds.jsonl / 经验库

外层 Task Manager (src/runner.ts) 负责一切可确定化的工程决策: VPN 连通性预检、选题调度 (多 flag 题优先, 按官方 tsec-benchmark SDK 语义)、worker 池并行 (默认 3, 对齐平台 3 容器上限)、容器生命周期管理、单题时限控制 (单 flag 35 分钟 / 多 flag 每 flag 20 分钟)、停滞检测 (连续 6 轮无进展触发 hint 提醒, 无人值守不弃题)、多 pass 重试 (未完成题在队列清空后重拉清单再战, findings/hint/flag 进度跨尝试继承)、超时告警升级给人类决策、turn 级硬超时。所有输出统一走 hooks.emit 事件流, CLI 与 Web UI 只是同一事件流的两种消费者。

内层 ReAct 循环 (src/agent.ts, 约 200 行) 是经典 while 循环: 流式生成回复 → 解析 tool_call → 执行工具 → 结果回灌上下文。内置 context compaction 机制: 消息超过 200 条时由 LLM 摘要压缩, 保留最近 60 条, 并有孤儿 tool_result 切点保护, 压缩后解题链状态 (chain state) 跨 compaction 持久化。

三层记忆 (src/state.ts):

- state.json: 每轮注入 system prompt 的结构化状态快照 (当前题目、已获 flag、尝试记录);
- rounds-<code>.jsonl: 全量可审计的轮次日志, 支撑赛后复盘与实验分析;
- experience/<domain>.jsonl: 跨题目持久化的经验库。

工具系统 (工具聚合层) 是 Agent 与靶场之间的全部能力出口, 采用「一个注册表、统一调用约定」的聚合设计 (src/pentools.ts), 分四部分:

- **内置基础工具:** run_bash (受限 shell)、write_file、edit (src/tools.ts), 覆盖通用的命令执行与文件操作;
- **benchmark 对接工具 (src/sec_tools.ts):** 提交 flag、journal 结构化汇报等, 与 TSec 平台语义对齐 (多 flag 进度以平台 correct_flag_count 为准, 重复提交幂等);
- **pentool 统一注册表:** 所有渗透工具——无论是 ffuf、nmap、nuclei、sqlmap、whatweb、ROPgadget、pwntools 这类镜像内置的开源二进制 (source: builtin), 还是蒸馏产出的 Python 脚本 (source: distilled, 存于 pentools/custom/) ——都以同构条目登记: {name, kind, command, purpose, usage, timeout}。LLM 只看到一张扁平的工具清单和一句话用途/用法示例, 通过统一的 pentool 工具入口按名调用, 输出经截断后回灌上下文。这种「声明式条目 + 单一入口」的设计让新增工具对 Agent 完全透明: 蒸馏产出一个新工具, 等于注册表里多一行 JSON, prompt、调度、UI 零改动。注册表读取带容错——

registry.json 被并发写坏 (torn read) 时自动回退到最近一次完好的副本，避免一次坏写击穿整个工具层；

- **工具蒸馏管道** (src/distill.ts)：Agent 解题时自写的临时脚本沉淀到 skills_staging/，积累到阈值（默认 5 个）后触发蒸馏——开启一个独立的 LLM 会话，逐个审阅 staging 脚本并结合近期 rounds.jsonl 的实战场景，把有共性的能力蒸馏成一个通用工具登记进注册表（一次只产一个，宁缺毋滥，质量不达标则明确报告本轮无产出）。蒸馏过程同样走事件流，Web UI 可实时围观。目前已产出 flask_session_forge (Flask session 伪造/弱密钥爆破)、xxe_read_probe (XXE 文件读取 + WAF 绕过) 等实战工具。

经验层解决「知识复用」，蒸馏层解决「能力复用」：前者让 Agent 记得怎么打，后者让 Agent 手里有打过的武器，二者共同构成跨场次的能力积累闭环。

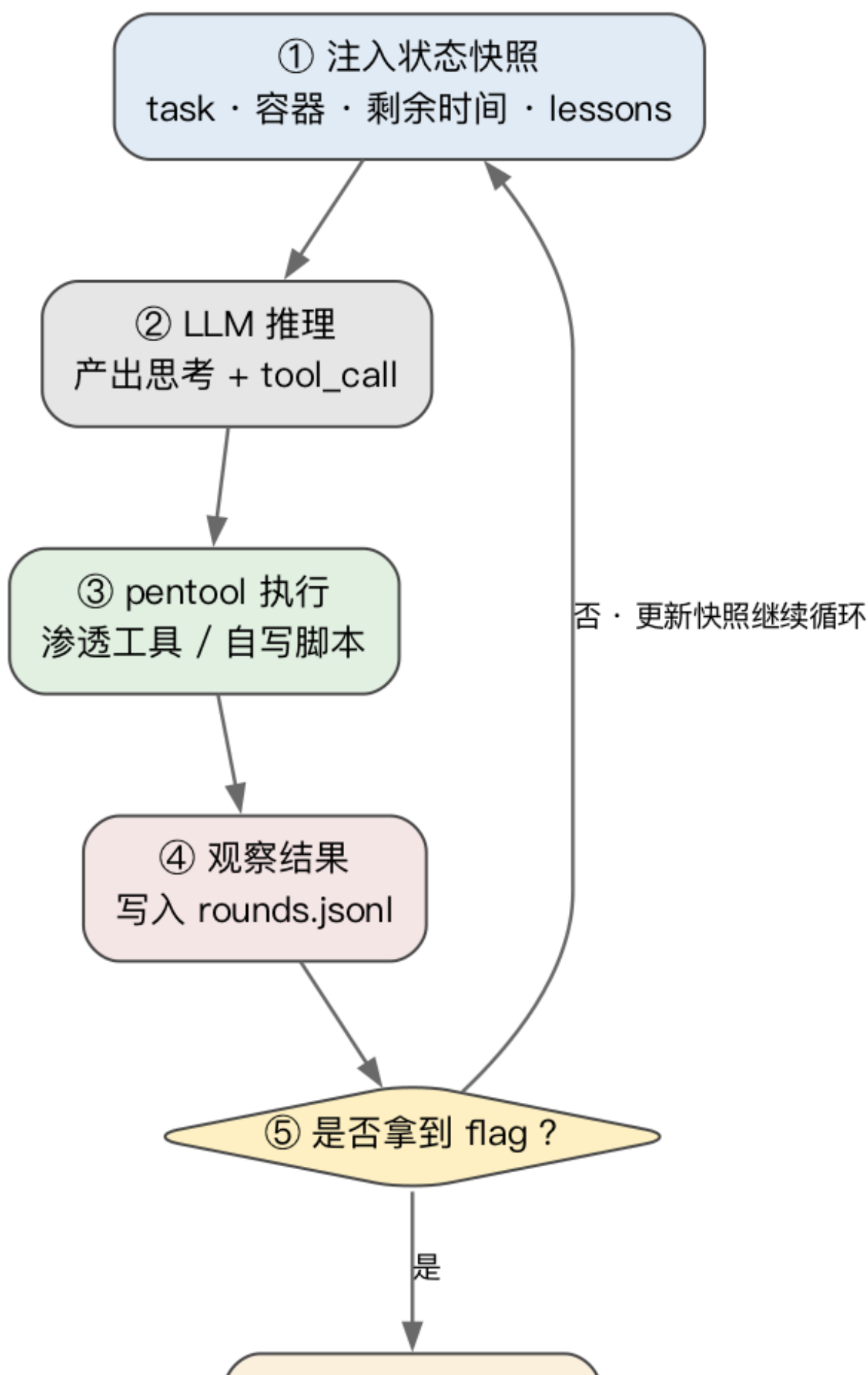
人机协作 (src/webui.ts + webui/) 提供实时事件流、状态面板、停滞告警决策（继续/换题/放弃）、指令注入 (inbox 队列) 和在线配置修改，使 Agent 在「无人值守」与「人在环路」两种模式间平滑切换。

知识库：playbooks/ 内置 ai-llm、blockchain、multistage、pwn、weaver-oa 五份领域渗透 playbook，作为先验知识注入。

3. 核心算法与方法

3.1 双层决策分离

LLM 擅长漏洞分析与利用构造，但不擅长预算管理 with 全局调度。系统将「做什么题、花多少时间、何时换题」交给确定性代码，将「怎么解题」交给 LLM，避免概率模型在长程任务上的预算失控。停滞检测以「连续 N 轮无实质性进展（无新 flag、无新发现）」为判据：无人值守时触发 hint 提醒并继续（题靠轮次/时限/deadline 收口，未完成题由后续 pass 重试），有人值守时升级给人类决策。



⑥ journal 汇报得分
进入下一题

3.2 上下文压缩 (Context Compaction)

长程渗透任务的消息数远超模型上下文窗口。当消息数超过 200 条时，由 LLM 对历史进行摘要压缩，仅保留最近 60 条原文。压缩时进行孤儿 tool_result 切点保护（避免截断在 tool_call 与其结果之间），并将解题链状态外置持久化，保证压缩不丢失关键中间结论。

3.3 经验沉淀与复用 (src/experience.ts)

赛后由 runner 以确定性方式（非 LLM）从轮次日志中提炼经验，按 web / pwn / ai-llm / blockchain / misc 五个领域存储为 JSONL，经验类型分 lesson（教训）、payload（有效载荷）、pitfall（坑）、pattern（模式）四类；重复经验不新增条目而是 score+1，下次运行时 renderForPrompt() 将各域 top N 经验注入 system prompt。这构成一个跨题目、跨场次的持续学习闭环，且不依赖模型权重更新。

3.4 工具蒸馏 (Tool Distillation)

蒸馏的具体管道见 2 节工具系统。其方法论要点在于：蒸馏由**独立 LLM 会话**执行，输入同时包含 staging 脚本原文与近期轮次日志（即脚本被写出来时的实战场景），并要求「一次只产一个、宁缺毋滥」——没有共性或质量不达标时允许零产出。这把「能力积累」从无脑堆脚本变成了有门禁的抽象过程，注册表只沉淀可复用的通用能力。

3.5 预算感知的多题调度

针对多 flag 题目优先调度，并按每 flag 时限做预算切片；配合按题统计的 token 计量 (tokens_by_challenge)，可对不同策略的性价比做量化对比。

4. 实验结果

在 TSec Benchmark 的一场完整实测中 (run-9494, 2026-08-15 15:53 至 20:30, 约 4.5 小时)，系统以 3 worker 并行调度覆盖 A/B/C/D/E/F 六大题类（Web 业务系统、防火墙穿透、大模型应用、云环境、评估对抗、固件/服务逆向），事件流共记录 235 条调度与得分事件：

- 总得分 **19025** 分，63 次得分事件，**成功解出 61 道不同题目**（含 B-02、B-03 两道多 flag 题各提交 2 个 flag）；
- 仅 2 题首次尝试失败（C-03、A-03），均经重启容器、调整策略后二次尝试解出；8 道题在停滞触发提示机制（hint 代价为分值的 10%），提示后全部转化为得分；
- 整体节奏呈「低分题速通、高分题攻坚」：E 类 250 分题平均每题约 2 分钟解出；C-02（智算模型托管引擎，450 分）经 4 次容器重启与提示后最终拿下，体现了停滞检测 + 提示升级

+ 重试调度组合策略的有效性；

- 典型难点攻坚：A-13（PyDash 原型链污染，450 分）在首次 15 分钟未果后暂停换题、重启环境并查看提示，二次进场 18 秒即解出，验证了「隔离放弃—换脑重试」机制对卡死场景的恢复能力。

5. 创新点总结

1. **确定性外层 + 概率性内层的双层 ReAct 架构**：将预算、时限、放弃、并行调度等工程决策从 LLM 手中剥离，用代码保证长程自主任务的可控性与可审计性。
2. **确定性经验提炼机制**：赛后经验提炼不经过 LLM，避免摘要幻觉污染经验库；经验按域分类、按分去重，构成低成本的持续学习闭环。
3. **工具自蒸馏**：Agent 把解题中自写的脚本蒸馏为正式工具，实现工具集的在线自扩展，是「Agent 自我能力积累」的一种轻量实现。
4. **带切点保护的上下文压缩 + 链状态外置持久化**：保证超长渗透链在压缩后仍不丢失关键中间结论。
5. **事件流统一抽象**：所有输出走同一事件流，CLI / Web UI / 告警系统均为消费者，天然支持人在环路的接管与干预。

6. 后续方向

- 自动蒸馏阈值与质量门禁的调优（当前 staging 阈值策略尚在手动/自动两版之间收敛）；
- 扩大 benchmark 样本量，统计经验库规模与得分率的相关性；
- 针对长耗时攻坚题（如 C-02、F2-05 类）研究子目标分解与更早的提示触发/放弃判据，进一步压缩无效探索时间。